

DevOps Engineering Foundation - 6-Week Intensive Bootcamp

WEEK 6: Monitoring, Final Project & Demo

Learning Outcomes:

- Implement monitoring and logging solutions
- Set up alerting for infrastructure and applications
- Complete and deploy final capstone project
- Present project demonstrating end-to-end DevOps workflow

Resources:

Monitoring & Logging:

- [Prometheus Tutorial for Beginners](#) by TechWorld with Nana (1 hour)
- [Grafana Tutorial for Beginners](#) by Geerling Guy (30 min)
- [Prometheus Documentation](#)
- [Grafana Documentation](#)

DevOps Best Practices:

- [DevOps Best Practices](#) by Simplilearn (1 hour)
- [The DevOps Handbook](#) - Key chapters

FINAL CAPSTONE PROJECT: Production-Ready E-Commerce Microservices Platform

Project Description:

Build and deploy a complete microservices-based e-commerce platform demonstrating every DevOps skill learned.

Architecture Components:

Application Services (Choose your stack):

1. Frontend Service: React/Vue.js shopping interface
2. Product Service: REST API for product catalog (Node.js/Python)
3. Cart Service: Shopping cart management
4. User Service: Authentication and user management
5. Order Service: Order processing
6. Database: PostgreSQL for relational data, Redis for caching

DevOps Requirements:

1. Version Control:

- Mono-repo or multi-repo strategy
- Feature branch workflow
- Protected main branch with PR requirements
- Conventional commit messages

2. Containerization:

- Dockerfile for each service
- Multi-stage builds for optimization
- Docker Compose for local development
- Private container registry

3. Infrastructure as Code:

- Terraform to provision:
 - AWS VPC with proper networking
 - EKS cluster or EC2 instances
 - RDS database
 - S3 for static assets
 - Load balancers
 - Auto-scaling groups

- Separate environments (dev, staging, prod)

4. Kubernetes Orchestration:

- Deployment manifests for all services
- ConfigMaps and Secrets management
- Services and Ingress configuration
- Horizontal Pod Autoscaling
- Resource limits and requests
- Namespaces for environment separation

5. CI/CD Pipeline:

- Automated testing (unit, integration)
- Security scanning (vulnerability checks)
- Docker image building and tagging
- Automated deployment to Kubernetes
- Blue-green or canary deployment strategy
- Rollback mechanisms

6. Monitoring & Logging:

- Prometheus for metrics collection
- Grafana dashboards showing:
 - Application performance metrics
 - Infrastructure health
 - Business metrics (orders, users)
- Centralized logging (ELK stack or CloudWatch)
- Alerting rules for critical issues
- Distributed tracing (optional: Jaeger)

8. Documentation:

- Architecture diagrams

- Setup and deployment instructions
- API documentation
- Runbooks for common operations
- Disaster recovery procedures

Deliverables:

Required Outputs:

1. GitHub Repository:
 - Complete source code
 - Infrastructure code
 - CI/CD configurations
 - Comprehensive README
2. Live Application:
 - Publicly accessible URL
 - Working functionality
 - SSL enabled
3. Monitoring Dashboard:
 - Live Grafana dashboard (public or screenshot)
 - At least 5 meaningful metrics
4. Documentation:
 - Architecture diagram
 - Deployment guide
 - Video walkthrough